Introduction to Natural Language Processing and Classical Language Models

Open in Colab

Part 1:- Introduction to Basic NLP Opertations

- Tokenization
- Normaliztion
- Stopwords Removal
- Stemming and Lemmatization
- Representing Text:
  - Bag of Words (BoW)
  - Term-Frequency Invesrse Term Frequency (TD/IDF)

Download NLTK from the Internet and install it on our PC

```
! pip install nltk
```

```
Requirement already satisfied: nltk in c:\users\me\myenv\lib\site-packages (3.9.1)
Requirement already satisfied: click in c:\users\me\myenv\lib\site-packages (from nltk) (8.1.8)
Requirement already satisfied: joblib in c:\users\me\myenv\lib\site-packages (from nltk) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in c:\users\me\myenv\lib\site-packages (from nltk) (2024.11.6)
Requirement already satisfied: tqdm in c:\users\me\myenv\lib\site-packages (from nltk) (4.67.1)
Requirement already satisfied: colorama in c:\users\me\myenv\lib\site-packages (from click->nltk) (0.4.6)
```

Import NLTK so that we can use it in our code

```

import nltk
```

Tokenize words using NLTK package

```
from nltk.tokenize import word_tokenize

# punkt_tab is a pretrained tokenization model used for splitting text into sentences and words.
nltk.download('punkt_tab')

sentence = "Large language models are revolutionizing business applications."
tokens = word_tokenize(sentence)
print(tokens)
```

```
['Large', 'language', 'models', 'are', 'revolutionizing', 'business', 'applications', '.']
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\me\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Normalization: Converting text to a standard form to reduce variability:

Convert all letters to lower case

```
normalized_tokens = [token.lower() for token in tokens]
print(normalized_tokens)
```

```
['large', 'language', 'models', 'are', 'revolutionizing', 'business', 'applications', '.']
```

Remove punctuation

```
import re

# [^\w\s] means any character that is not a word character or whitespace, ^ inside square brackets negates the expression.
```

```
# \w is a word character (alphanumeric character plus underscore)
normalized_tokens = [re.sub(r'[^\w\s]', '', token.lower()) for token in tokens]
print(normalized_tokens)
```

```
['large', 'language', 'models', 'are', 'revolutionizing', 'business', 'applications', '']
```

[Click here to open regular expressions tutorial]

Stopword Removal:

Eliminating common words that add little meaning

First, we need to download the stopwords dataset and import it in our code

```
nltk.download('stopwords') #stopwords are common words that are often removed from text as they are not useful for analysis.
from nltk.corpus import stopwords

nltk.download('stopwords')
stop_words = set(stopwords.words('english'))

print("These words are removed from the text: ", stop_words)
```

```
These words are removed from the text:  {"he's", 'her', 'an', 'didn', 'for', 'ours', 'of', 'ain', 'his', 've', "doesn't", 'h
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\me\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\me\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Then we can remove stopwords from our sentence

```
filtered_tokens = [token for token in normalized_tokens if token and token not in stop_words]
print(filtered_tokens)
```

```
['large', 'language', 'models', 'revolutionizing', 'business', 'applications']
```

Stemming vs. Lemmatization in NLP

Stemming Example

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stemmed_tokens = [stemmer.stem(token) for token in filtered_tokens]

print(stemmed_tokens)
```

```
['larg', 'languag', 'model', 'revolution', 'busi', 'applic']
```

Lemmatization Example

```
#nltk.download('wordnet')

from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
lemmatized_tokens = [lemmatizer.lemmatize(token) for token in filtered_tokens]

print(lemmatized_tokens)
```

```
['large', 'language', 'model', 'revolutionizing', 'business', 'application']
```

Representing Text - Bag of Words and TF-IDF

Bag of Words (BoW)

A simple way to represent text as numerical vectors by counting word occurrences and representing each sentence as a vector or word counts

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "Large language models revolutionize business.",
    "Business applications benefit from AI.",
    "Language models learn from text data."
]

#  CountVectorizer is used to convert a collection of text documents to a matrix of token counts.
#  The output is a sparse matrix where each row represents a document and each column represents a word in the corpus.
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

# Get feature names and the array representation of the matrix
feature_names = vectorizer.get_feature_names_out()

df = pd.DataFrame(X.toarray(), columns=feature_names)
df
```

	ai	applications	benefit	business	data	from	language	large	learn	models	revolutionize	text
0	0	0	0	1	0	0	1	1	0	1	1	0
1	1	1	1	1	0	1	0	0	0	0	0	0
2	0	0	0	0	1	1	1	0	1	1	0	1

Term Frequency-Inverse Document Frequency (TF-IDF)

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "Large language models revolutionize business.",
    "Business applications benefit from AI.",
    "Language models learn from text data."
]

tfidf_vectorizer = TfidfVectorizer()

# Transform the corpus into a document-term matrix
# Each row represents a document in the corpus
# Each column represents a unique word in the corpus
X_tfidf = tfidf_vectorizer.fit_transform(corpus)


df = pd.DataFrame(X_tfidf.toarray(), columns=tfidf_vectorizer.get_feature_names_out())
df
```

	ai	applications	benefit	business	data	from	language	large	learn	models	revolutionize	text
0	0.000000	0.000000	0.000000	0.393511	0.000000	0.000000	0.393511	0.51742	0.000000	0.393511	0.51742	0.000000
1	0.490479	0.490479	0.490479	0.373022	0.000000	0.373022	0.000000	0.00000	0.000000	0.000000	0.00000	0.000000
2	0.000000	0.000000	0.000000	0.000000	0.459548	0.349498	0.349498	0.00000	0.459548	0.349498	0.00000	0.459548

Practical Example

Sentiment Analysis of Movie Reviews

This application will go through the entire text preprocessing pipeline and show how it contributes to a real-world NLP task. Setup

Dataset: Use a small dataset of movie reviews (positive and negative) - you could use a subset of IMDB reviews or create 10-15 simple examples. Visual Flow: Create a slide that shows the entire pipeline:

Text Processing Pipeline

Raw Text → Tokenization → Normalization → Stop Words Removal → Stemming/Lemmatization → BoW → Classification

First, we need to download all the required modules and data process our text

```
# Download necessary NLTK data
nltk.download('punkt') # required for tokenization
nltk.download('stopwords') # required for removing stopwords
nltk.download('wordnet') # required for lemmatization
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\me\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\me\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\me\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

Next, we will do the following:

- Import the needed libraries
- Download any additional modules that are arequired
- Prepare the dataset i.e Text Corpus

```
import pandas as pd
import re
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# Sample movie reviews dataset
reviews = [
    {"text": "This movie was absolutely fantastic! Great acting and storyline.", "sentiment": 1},
    {"text": "I loved this film. The characters were so well developed.", "sentiment": 1},
    {"text": "Amazing cinematography and directing. One of the best films I've seen.", "sentiment": 1},
    {"text": "The acting was good but the story was too predictable.", "sentiment": 0},
    {"text": "Terrible movie. I wasted two hours of my life.", "sentiment": 0},
    {"text": "The special effects were amazing but everything else was boring.", "sentiment": 0},
    {"text": "I enjoyed the action sequences but the dialogue was poorly written.", "sentiment": 0},
    {"text": "Brilliant performance by the lead actor! Highly recommended.", "sentiment": 1},
    {"text": "So disappointing. The trailer was better than the actual movie.", "sentiment": 0},
    {"text": "A masterpiece of modern cinema. I was captivated throughout.", "sentiment": 1}
]

# Create DataFrame
df = pd.DataFrame(reviews)
print("Original Data:")
print(df.head())
print("\n")
```

```
Original Data:
                                                text  sentiment
0  This movie was absolutely fantastic! Great act...          1
1  I loved this film. The characters were so well...          1
2  Amazing cinematography and directing. One of t...          1
3  The acting was good but the story was too pred...          0
4     Terrible movie. I wasted two hours of my life.          0
```

In the text segment below, we will define the required text processing functions:

1. tokenize text function
2. normalize_tokens function
3. remove_stopwords function
4. stem_tokens function
5. lemmatize_tokens function
6. preprocess_text function, the pipeline that calls all the previous functions

```
# Step 1: Tokenization
def tokenize_text(text):
    tokens = word_tokenize(text)
    # print(f"Sentence after tokenization: {tokens}") # uncomment if need to see results
    return tokens

# Step 2: Normalization
def normalize_tokens(tokens):
    # Convert to lowercase and remove punctuation
    normalized = [re.sub(r'[^\w\s]', '', token.lower()) for token in tokens]
    normalized = [token for token in normalized if token]  # Remove empty strings
    # print(f"Tokens after normalization: {normalized}") # uncomment if need to see results
    return normalized

# Step 3: Remove stop words
def remove_stopwords(tokens):
    stop_words = set(stopwords.words('english'))
    filtered = [token for token in tokens if token not in stop_words]
    # print(f"Tokens after stopword removal: {filtered}") # uncomment if need to see results
    return filtered

# Step 4a: Stemming
def stem_tokens(tokens):
    stemmer = PorterStemmer()
    stemmed = [stemmer.stem(token) for token in tokens]
    # print(f"Tokesn after stemming: {stemmed}") # uncomment if need to see results
    return stemmed

# Step 4b: Lemmatization
def lemmatize_tokens(tokens):
    lemmatizer = WordNetLemmatizer()
    lemmatized = [lemmatizer.lemmatize(token) for token in tokens]
    # print(f"Tokens after lemmatization: {lemmatized}") # uncomment if need to see results
    return lemmatized

# Complete preprocessing pipeline
def preprocess_text(text, use_stemming=True):
    tokens = tokenize_text(text)
    normalized_tokens = normalize_tokens(tokens)
    no_stopwords = remove_stopwords(normalized_tokens)

    if use_stemming:
        return stem_tokens(no_stopwords)
    else:
        return lemmatize_tokens(no_stopwords)

# Demonstrate the preprocessing pipeline on one example
print("PREPROCESSING PIPELINE DEMONSTRATION:")
sample_text = df['text'][0]
print(f"Original text: '{sample_text}'")
processed_tokens = preprocess_text(sample_text, use_stemming=True)
print(f"Processed tokens: {processed_tokens}")
print("\n")
```

```
PREPROCESSING PIPELINE DEMONSTRATION:
Original text: 'This movie was absolutely fantastic! Great acting and storyline.'
Processed tokens: ['movi', 'absolut', 'fantast', 'great', 'act', 'storylin']
```

Apply the text processing pipeline on all our sentences (entire text corpus)

```
df['processed_text'] = df['text'].apply(lambda x: ' '.join(preprocess_text(x, use_stemming=True)))
df
```

	text	sentiment	processed_text
0	This movie was absolutely fantastic! Great act...	1	movi absolut fantast great act storylin
1	I loved this film. The characters were so well...	1	love film charact well develop
2	Amazing cinematography and directing. One of t...	1	amaz cinematographi direct one best film seen
3	The acting was good but the story was too pred...	0	act good stori predict
4	Terrible movie. I wasted two hours of my life.	0	terribl movi wast two hour life
5	The special effects were amazing but everythin...	0	special effect amaz everyth els bore
6	I enjoyed the action sequences but the dialogu...	0	enjoy action sequenc dialogu poorli written
7	Brilliant performance by the lead actor! Highl...	1	brilliant perform lead actor highli recommend
8	So disappointing. The trailer was better than ...	0	disappoint trailer better actual movi
9	A masterpiece of modern cinema. I was captivat...	1	masterpiec modern cinema captiv throughout

Convert text into numerical representation using the Bag of Words Method, we will the use the simple count approach

```
# Bag of Words representation
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['processed_text'])

print("BOW Matrix (First 3 Documents):")
bow_df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
bow_df
```

BOW Matrix (First 3 Documents):

	absolut	act	action	actor	actual	amaz	best	better	bore	brilliant	...	special	stori	storylin	terribl	throughout
0	1	1	0	0	0	0	0	0	0	0	...	0	0	1	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2	0	0	0	0	0	1	1	0	0	0	...	0	0	0	0	0
3	0	1	0	0	0	0	0	0	0	0	...	0	1	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	1	0
5	0	0	0	0	0	1	0	0	1	0	...	1	0	0	0	0
6	0	0	1	0	0	0	0	0	0	0	...	0	0	0	0	0
7	0	0	0	1	0	0	0	0	0	1	...	0	0	0	0	0
8	0	0	0	0	1	0	0	1	0	0	...	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	1

10 rows × 51 columns

Create the classifier and train it on the generated BoW representations

```
X_train, X_test, y_train, y_test = train_test_split(X, df['sentiment'], test_size=0.3, random_state=42)

# Train a Naive Bayes classifier
clf = MultinomialNB()
clf.fit(X_train, y_train)
```

```
▾ MultinomialNB
MultinomialNB()
```

Use the trained classifier to make predictions using the test data (sentences that was not seen by the classifier during training)

```
y_pred = clf.predict(X_test)
```

Evaluate the classification results (Model Performance)

```
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.67

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.50      0.67         2
           1       0.50      1.00      0.67         1

    accuracy                           0.67         3
   macro avg       0.75      0.75      0.67         3
weighted avg       0.83      0.67      0.67         3
```

Part 2:- Language Models

Building a Simple N-gram Language Model

Import the required libraries

```
import random
from collections import defaultdict
```

Define the function that builds the language model

```
def build_ngram_model(text, n=2):
    """Build an n-gram language model from text."""
    tokens = word_tokenize(text.lower())
    ngrams_dict = defaultdict(list)

    # Create dictionary of n-grams and possible next words
    for i in range(len(tokens) - n):
        current_ngram = tuple(tokens[i:i+n])
        next_word = tokens[i+n]
        ngrams_dict[current_ngram].append(next_word)

    return ngrams_dict
```

Define the function that is used to generate new text based on an input seed text

```
def generate_text(model, seed, length):
    """Generate text using the n-gram model."""
    current = seed
    result = list(seed)

    for _ in range(length):
        if current in model:
            # Randomly select a possible next word
            next_word = random.choice(model[current])
            result.append(next_word)
            # Update current n-gram
            current = current[1:] + (next_word,)
        else:
            # If current n-gram is not in model, break
            break

    return ' '.join(result)
```

Create a sample corups to build our language model

```
# Sample text corpus
corpus = """Large language models are transforming how businesses operate.
These models can understand language, generate text, and perform various tasks.
```

```
Businesses use language models for customer service, content creation, and data analysis.
Language models learn patterns from vast amounts of text data."""

# Build a bigram model
bigram_model = build_ngram_model(corpus, 2)

print("Bigram Dictionary/Model:")
for key, value in bigram_model.items():
    print(f"{key} → {value}")
```

```
Bigram Dictionary/Model:
('large', 'language') → ['models']
('language', 'models') → ['are', 'for', 'learn']
('models', 'are') → ['transforming']
('are', 'transforming') → ['how']
('transforming', 'how') → ['businesses']
('how', 'businesses') → ['operate']
('businesses', 'operate') → ['.']
('operate', '.') → ['these']
('.', 'these') → ['models']
('these', 'models') → ['can']
('models', 'can') → ['understand']
('can', 'understand') → ['language']
('understand', 'language') → [',']
('language', ',') → ['generate']
(',', 'generate') → ['text']
('generate', 'text') → [',']
('text', ',') → ['and']
(',', 'and') → ['perform', 'data']
('and', 'perform') → ['various']
('perform', 'various') → ['tasks']
('various', 'tasks') → ['.']
('tasks', '.') → ['businesses']
('.', 'businesses') → ['use']
('businesses', 'use') → ['language']
('use', 'language') → ['models']
('models', 'for') → ['customer']
('for', 'customer') → ['service']
('customer', 'service') → [',']
('service', ',') → ['content']
(',', 'content') → ['creation']
('content', 'creation') → [',']
('creation', ',') → ['and']
('and', 'data') → ['analysis']
('data', 'analysis') → ['.']
('analysis', '.') → ['language']
('.', 'language') → ['models']
('models', 'learn') → ['patterns']
('learn', 'patterns') → ['from']
('patterns', 'from') → ['vast']
('from', 'vast') → ['amounts']
('vast', 'amounts') → ['of']
('amounts', 'of') → ['text']
('of', 'text') → ['data']
('text', 'data') → ['.']
```

Use the langauge model we built to predict new words base on an input text

Make sure to run the code below several times to see how the language model generates new text everytime (**A Non-Deterministic Stocastich Process**)

```
# Generate text using the model
seed = ('language', 'models')
generated_text = generate_text(bigram_model, seed, 20)
print(generated_text)
```

```
language models learn patterns from vast amounts of text data .
```

Evaluating Language Models: Perplexity

Perplexity measures how well a language model predicts a sample:

```
import numpy as np
from collections import Counter
```

```python

def get_bigram_probability(model, unigram_counts, vocab_size, word1, word2):
    """Calculate the probability of a bigram."""
    bigram = (word1, word2)
    bigram_count = len(model[bigram])
    unigram_count = unigram_counts[word1]
    # Add-one smoothing
    probability = (bigram_count + 1) / (unigram_count + vocab_size)
    return probability

def calculate_perplexity(test_text, model, unigram_counts, vocab_size):
    """Calculate perplexity of test text using the bigram model."""
    tokens = word_tokenize(test_text.lower())
    log_probability = 0

    for i in range(len(tokens) - 1):
        bigram = (tokens[i], tokens[i+1])
        probability = get_bigram_probability(model, unigram_counts, vocab_size, tokens[i], tokens[i+1])
        log_probability += np.log2(probability)

    # Perplexity = 2^(-average log probability)
    perplexity = 2 ** (-log_probability / (len(tokens) - 1))
    return perplexity

# Calculate unigram counts and vocabulary size
tokens = word_tokenize(corpus.lower())
unigram_counts = Counter(tokens)
vocab_size = len(unigram_counts)

# Test the model on new text
test_text = "Language models help businesses understand customer feedback."
perplexity = calculate_perplexity(test_text, bigram_model, unigram_counts, vocab_size)
print(f"Perplexity: {perplexity:.2f}")
```

```
Perplexity: 28.45
```